LOOP-3 SUBMISSION DRAFT — DERIVATION OF RECORD

From the Qn Universe to QnIR v0.3

The repair sequence retraced from the frozen minimal core through the conformance-oracle specification; the artifact-share adjudication and the principal's rulings registered; and the open seed-language question — Rust, WAT (wasm/wasi), or lower assembly — put to the Loop-3 review before the QnLang design.

Nguyen, C. D. Q. — program principal

Qn universe program · derivation compiled under the BOC policy of docs/simemp.md §7 · repository bytes cited, not inlined

Table of Contents

1. The Program of Record and Its Method 2

1.1 Purpose of This Document . . . 2

1.2 The Method: BOC Discipline End to End . . . 2

1.3 The Two Loops That Produced This Program . . . 3

2. The Repair Sequence Retraced 4

2.1 Step 1 — The Frozen Minimal Core (v0.3.0-draft.1) . . . 5

2.2 Step 2 — Operational Semantics (v0.3.0-draft.2) . . . 5

2.3 Step 3 — Work Accounting (v0.3.0-draft.3) . . . 6

2.4 Step 4 — The Conformance Oracle, Specification Share (v0.3.0-draft.4) . . . 6

3. The Artifact Share: Adjudication, Rulings, and Protocols 7

3.1 The Candidate Scan for the First Executable Increment . . . 7

3.2 The Principal's Rulings, Registered . . . 8

3.3 The Declared Shape of the First Artifact Increment . . . 9

3.4 Sequencing with Loop 3 . . . 10

4. The Open Question: The Seed-Language BOC before QnLang Design 10

4.1 What a Seed Is, and Why the Question Precedes QnLang . . . 10

4.2 Governing Criteria, Derived from the Committed Specification . . . 11

4.3 The Candidate Set and Preliminary Input . . . 12

4.4 The Questions Put to the Review . . . 13

5. Governance State and Scope of This Submission 14

5.1 The Standing Governance Queue . . . 14

5.2 Review Program State and the Draft's Status . . . 15

5.3 What This Submission Changes, and What It Does Not . . . 15

6. What Is Asked of the Loop-3 Review 15

6.1 Scope of Review . . . 15

6.2 Output Convention . . . 16

6.3 Independence and the Limitations Statement . . . 16

6.4 After the Verdict . . . 16

References 17

1. The Program of Record and Its Method

1.1 Purpose of This Document

This document is the derivation of record for QnIR v0.3, prepared as the narrative submission of the Loop-3 adversarial review. It retraces every committed increment of the v0.3 program from the Loop-2 verdict to the present state: the four committed draft increments of `docs/qnir.md` (the frozen minimal core, the operational semantics, the work accounting, and the conformance-oracle specification), the artifact-share adjudication that selects the first executable increment, the principal's toolchain and placement rulings, and the open seed-language question that this submission formally puts to the review before any QnLang design begins. Each claim below is cited to committed repository bytes by path, version, and section anchor, so that the reviewer holding the repository dump can verify every statement against the exact bytes it names.

Unlike the retired v0.2.2 submission — a 187-page PDF that inlined the repository's markdown contents as annexes — this document inlines nothing. The v0.2 review's own limitation statement recorded the reason such inlining must end: repository bytes, annex round trips, benchmarks, proof artifacts, and validator behavior were not independently reproduced, and any data loss between the live repository and the reviewed PDF was inherited by the review's conclusions as unexamined inaccuracy. Citation replaces inlining here, and the accompanying repository dump is the annex. This removes the data-loss class entirely: the reviewer audits the same bytes the program commits, the derivation narrative only maps them, and every checkable claim the promotion criterion names becomes checkable against the dump the reviewer already holds.

1.2 The Method: BOC Discipline End to End

The program runs under the BOC (Best of Choices) policy of `docs/simemp.md` §7. Every step is adjudicated before it is executed: a Loop-1 candidate scan enumerates the defensible options with no dead entries, and a Loop-2 lexicographic π adjudication orders the survivors by Efficiency, then Modularity, then Portability. Increments are drafted one file at a time, additively (Axiom 13 — no existing content is mutated in place), with any companion-file edits budgeted up front in the wave record so that no unbudgeted-companion flag can recur. Drafting is separated from verification, and verification is separated from commitment: the drafting agent never commits, the principal validates and commits, and the live site is confirmed afterwards. The gates below have applied to every wave retraced in this document.

Gate	What it verifies	Performed by
Adjudication	Loop-1 candidate scan (no dead entries) and Loop-2 π order: Efficiency, then Modularity, then Portability	Drafting agent; principal green-lights
Drafting	One file at a time; additive-only edits; budgeted companions declared in the wave record	Drafting agent
Byte-level verification	Structural checks: anchors, include parity, version-field consistency; for executable artifacts, behavioral checks added (Chapter 3)	Drafting agent
Validation and commit	Principal validates the presented diff shape, commits to the repository, confirms the live render	Principal
Registration	Wave record, forward map, and registers updated; no queue trigger fired silently	Drafting agent, cited to committed bytes

Table 1 — The program's standing protocol gates

1.3 The Two Loops That Produced This Program

The framework's specifications are stress-tested through an external adversarial review program: major draft artifacts are submitted to independent AI reviewers, and the verbatim returned transcripts are committed as artifacts of record under `docs/AiReviews/`. Two loops are complete to date, per `docs/index.md` v1.14.0. Loop 1 (Google Deep Review, October 2026) reviewed the condensed Qn universe specification together with the QnIR v0.1 candidate draft and established the open-question protocol under which subsequent loops run. Loop 2 (Kimi K3.1-Pro, 27 October 2026) reviewed the QnIR draft v0.2.2 — the retired 187-page derivation PDF — and returned the verdict "major revision; do not promote QnIR v0.2 to the canon."

Loop 2's returned artifact, committed verbatim as `docs/AiReviews/AiReviewQnIRv0.2.md`, is the direct ancestor of everything retraced in Chapter 2. It delivered eight numbered findings (F1 through F8), seventeen answers in the open-question registry (QO-16 through QO-32), a sixteen-objection grading table, and the required v0.3 repair sequence — six steps whose order is itself normative, since later artifacts inherit earlier definitions. The review also fixed the promotion criterion that governs canon entry, quoted verbatim at the end of Chapter 2, and recorded the limitation that shapes this submission's citation-based form.

8	17	16	6
numbered findings F1–F8	answers QO-16 – QO-32	objections graded	repair steps mandated

2. The Repair Sequence Retraced

The repair sequence of `AiReviewQnIRv0.2.md` §5 is the operative instruction set of the v0.3 program, and its own order is normative: later artifacts inherit earlier definitions. The table below preserves each step's acceptance condition verbatim, condensed only in layout, and records the discharge state as of this submission. The four discharged specification steps are then retraced individually, each citing the committed section of `docs/qnir.md` that discharges it. Steps 5 and 6 remain forward of the draft and are listed in the forward map of `qnir.md` §5 exactly as committed.

Step	Acceptance condition (verbatim)	State
1 — Freeze a minimal core	Constants, tuples, exact arithmetic, comparison, bounded structured control, status, commit, witness, and halt. Move calls, saturation, containers, fork–join, and road proofs to extensions.	Discharged (§1, draft.1)
2 — Publish operational semantics	Typed configurations, small-step transitions, block arguments, dynamic instances, token lifecycle, failure behavior, and deterministic encodings. State determinism as Eval(Intent, D, V) = Execution with all semantic versions explicit.	Discharged (§2, draft.2)
3 — Repair work accounting	One ledger equation; static operand bounds; sound control-flow budget admission; reserve/charge/refund invariants; no double billing.	Discharged (§3, draft.3)
4 — Build a conformance oracle	Assembler, validator, interpreter, and independent second implementation. Re-encode all examples and add adversarial tests for loops, calls, collisions, unresolved statuses, and budget boundaries.	Specification share discharged (§4, draft.4); artifact share registered to follow-on increments
5 — Strengthen commitments	Full-transition trace roots, collision-safe artifact comparison, normative lineage, and versioned verbalizer/extractor identities.	Forward
6 — Add extensions with evidence	Container BOC, call trade study, liveness histograms, choreographic refinement, then class-specific AIRs and measured Realization A/B break-even.	Forward

Table 2 — The six-step v0.3 repair sequence (acceptance conditions verbatim; state as of v0.3.0-draft.4)

2.1 Step 1 — The Frozen Minimal Core (v0.3.0-draft.1)

The freeze is executed as a closed inventory at construct-family level: nine families — constants, tuples, exact arithmetic, comparison, bounded structured control, status, commit, witness, and halt — form the entire instruction domain of the core, and anything outside them has no semantics rather than an improvised one (`qnir.md` §1.2). Alongside the inventory stands an explicit deferral register of five families — calls, saturation, containers, fork–join, and road proofs — which is the priced extension agenda, not a silence: each deferred construct is named, and each will be admitted only with evidence at Step 6 (`qnir.md` §1.3). A constraints-inherited register then binds the obligations the v0.2 review attached to the core: F1's declared-bound discipline, the F7 re-encoding obligation (every example re-encoded against the frozen core only, by a normative assembler and validator), the QO-22 token-lifecycle test list, and the family-9 fatal-terminal rows (`qnir.md` §1.4). The Step-1 acceptance condition of the sequence is met by construction at commit, and the freeze has remained byte-unchanged through every later increment.

2.2 Step 2 — Operational Semantics (v0.3.0-draft.2)

Step 2 publishes the determinism statement `Eval(Intent, D, V) = Execution` with all three version components explicit: the semantics version pins the transition relation, the lifecycle algebra, and the terminal-wins rule; the encoding version pins the layout of the deterministic encodings; the profile version pins the declared ceilings and bounds that ride Intent (`qnir.md` §2.1). Versions are part of the determinism claim, not metadata about it — a change to any component is a different `Eval`. Execution operates on typed ten-component configurations through a small-step relation in which at most one successor configuration exists per step, with the clock advancing one tick per transition (`qnir.md` §2.2–§2.3). Block arguments and dynamic instances discharge F3; the token lifecycle algebra with its two tiers and the terminal-wins rule discharges QO-22, so that failure behavior is typed, sited, and never silently absorbed.

The deterministic encodings of §2.6 preserve the 64-bit word as transport and commitment unit (QO-30) while making values exact: an exact value encodes as its declared type tag, a word-length prefix, and the minimal big-endian sequence of 64-bit magnitude words under a declared sign convention — no leading zero words, one canonical zero — so each value has exactly one encoding. Non-uniform per-instruction attributes ride the EXT-flag pattern through a canonical side table carried by Intent, and every transition commits the F5 field list under domain-separated, CBOR-disciplined (RFC 8949) encoding, so equal records are byte-equal records. The fixed-width/exact reconciliation of §2.7 then resolves the tension with the inherited "small, fixed-width deterministic machine" description by declaration: exact value semantics over a fixed-width transport, a stance forced by Obj 2 — no universal capacity constant may become a core validity limit — rather than preferred by taste.

2.3 Step 3 — Work Accounting (v0.3.0-draft.3)

Step 3 commits the one ledger equation (F2): the published ledger is a derived record, equal to the reconstruction from the transition records' charged and refunded fields plus the terminal receipt's token itemization — a declaration that makes ledger checking mechanical by construction, since equal executions have byte-equal ledgers and no independent arithmetic exists anywhere in the check (`qnir.md` §3.1). Static operand bounds form the cost domain (F1 discharge, §3.2), and reserve, charge, and refund are typed ledger movements with the no-double-billing invariant (§3.3). Sound control-flow budget admission (F4 discharge, §3.4) gates the execution's first attempted transition on `W_budget ≥ B(P)`, where `B(P)` is the declared maximum-work bound of the program: the specification defines `B(P)` and deliberately no algorithm, registering the computational surrogate to Step 4 with a declared safe error direction.

Two bookkeeping facts from this wave matter to the review. First, the Step-3 acceptance condition records that the Steps 1–3 minimum named by the Adversarial Review Program's Loop-3 gate is met as of v0.3.0-draft.3 (`qnir.md` §3.6) — the gate this submission now exercises. Second, the wave carried its budgeted companion: a one-line additive extension of the QnIR Draft entry's description in `docs/_data/navigation.yml`, declared up front in the wave record, committed alongside the draft, and confirmed by the principal — the pattern every later wave has repeated.

2.4 Step 4 — The Conformance Oracle, Specification Share (v0.3.0-draft.4)

Step 4's specification share, committed as §4 of `qnir.md` at v0.3.0-draft.4, declares the mechanism that makes the promotion criterion's claims checkable rather than asserted. The differential protocol (§4.1) requires two implementations of the transition relation — the reference interpreter and the second implementation — built as separate codebases from the committed bytes of §1–§3 and from nothing else: no shared code, no shared derivation, and no common input beyond the committed text and the declared vector format. Conformance is decided by byte-equality of outputs at five named agreement points (canonical value encodings, the ordered transition-record sequence, pre/post state roots, the terminal receipt, and the reconstructed ledger), never by code review; the protocol adjudicates the disagreement, never the implementation.

The normative requirements for the four artifacts follow (§4.2–§4.3): an assembler whose input is an example's source form over the nine frozen families and whose output is the canonical 64-bit word encoding of §2.6, deterministic under the same discipline as the encodings it produces; a validator enforcing the four declared Intent-rejection classes, the static operand bounds, and the budget-admission rule, with the `B(P)` surrogate's declared error direction — false rejection, never false admission, with any under-approximation caught by the fuel backstop at run time; and the reference interpreter implementing the §2.3 relation exactly, together with the state-root function artifacts §2.6 named. The vector suite (§4.4) fixes the declared vector format and five adversarial families — loops, calls at the extension boundary, collisions, unresolved statuses, and budget boundaries — with the fatal-terminal vectors as first-class citizens and the per-class ledger traces of F2 published alongside the re-encoded examples. The acceptance condition (§4.5) is exact about what remains: the artifact share is discharged only when each named artifact exists as a repository increment that passes byte-level differential execution over the full suite — exactly then, and not before, the promotion criterion's two

checkable claims become checkable against repository bytes. The increment's wave record also declares its four forced single-token edits (all bookkeeping: two forward-map cross-references, the standing-queue cross-reference, and the semantics-version self-reference), none a change to any declared rule, encoding, or register.

> *Promotion criterion (verbatim,* `qnir.md`*): "v0.3 may enter the canon only when the core semantics and ledger pass independent differential execution, every normative example validates, and each retained extension has an explicit proof or measurement artifact. Until then, 'Candidate 1' must not be called Theorem 1, and R11', R12', R19, and the O(polylog) verification statement must remain conditional."*

3. The Artifact Share: Adjudication, Rulings, and Protocols

3.1 The Candidate Scan for the First Executable Increment

With the specification share committed, the program's category shifts from specification text to executable bytes, and the BOC adjudication moved with it: the question became the build order and shape of the first artifact. The Loop-1 scan enumerated eight candidates and discharged each with a recorded reason, so that no entry dies silently; the Loop-2 π order then confirmed the single survivor on all three axes. The survivor is the normative assembler: it is immediately testable against committed bytes, it produces the encoded-Intent material every later increment consumes, it sits at the pipeline head with zero upstream code dependencies, and its std-only single-file form keeps the second implementation's independence untouched — the second implementation conforms to §4.2's committed requirements, never to the assembler's code.

Candidate	Disposition
A — Normative assembler first	Selected. Pipeline head; zero upstream code dependencies; produces the encoded-Intent material all later increments consume; π-positive on Efficiency, Modularity, and Portability.
B — Vector suite first	Rejected: inverts the declared normativity direction — §4.2 makes the assembler the normative re-encoder, so the vectors' encoded-Intent fields must come from it, not the reverse; duplicates §2.6 hand-encoding per vector.
C — Reference interpreter first	Rejected: no upstream artifact exists to produce validated encoded Intents; the increment would sit unverifiable until everything else lands.
D — Validator first	Rejected: its test material is encoded Intents; without the assembler the fixtures are hand-encoded (the same inversion as B), and the B(P) surrogate's budget-boundary vectors cannot be exercised.
E — Second implementation first	Rejected: §4.1 conformance is decided at the agreement surface, which requires the reference side to differ against.
F — Per-class ledger traces first	Rejected: §4.4 fixes their site — published as conformance artifacts alongside the re-encoded examples; without the re-encoding the publication has nothing to sit alongside.
G — Hold main line for Loop 3	Not executable by the drafting agent: the loop's opening is the review program's own act. Recorded and flagged, never a dead entry; superseded when the principal proposed the review (§3.4).
H — Plan or manifest increment	Rejected: everything it would declare is already committed in §4.5 (each artifact its own file, citing §4, own version bindings); a restatement increment is budget waste.

Table 3 — Loop-1 candidate scan for the first artifact increment (no dead entries)

3.2 The Principal's Rulings, Registered

Three rulings by the program principal are now part of the wave record and bind the artifact share. First, placement: executable code lives under `src/`, not under `docs/` — a .py or .rs artifact is executable code, and the documentation tree is for specification and prose. The location is cleanly declarable because §4 pins no artifact paths; the citation direction runs from artifact to spec, so the first increment declares `src/qnir_assemble.rs` as its location and the corpus it re-encodes, citable by repository bytes. Second, language: Rust, not Python. The principal's rationale is performance and industry direction — frontier AI laboratories, Microsoft, and Google have re-coded performance-critical tools from Python or C/C++ into Rust — and the π re-score under Rust confirms the ruling: native-speed differential execution on exactly the hot path the oracle spends its time in (transition records, state roots, exact-integer arithmetic), a single `rustc`-compiled source file that literally preserves the one-file-at-a-time discipline, and static binaries with determinism by construction (explicitly ordered iteration, checked arithmetic, no floating point, no locale or time in output).

Third, the execution-verification protocol: for executable artifacts, byte-level verification alone is not sufficient, because §4.2's own claims are behavioral — equal inputs must produce byte-equal encodings. Artifact waves therefore add behavioral checks to the standing protocol: the artifact's self-test is executed and must exit cleanly; determinism is asserted by encoding the declared corpus twice and comparing byte-for-byte; the corpus re-encoding is emitted as the F7 evidence; and examples that cannot be expressed within the frozen families are verified to route to extension-boundary classification rather than being forced into the core. The principal's validation step correspondingly includes re-executing the declared command and comparing output before commit — the program's first independent reproduction of artifact behavior, the exact inverse of the v0.2 limitation this program exists to close. The toolchain fact is registered: rustc and cargo 1.99.1 were installed and verified in the drafting workspace on 29 October 2026, so the behavioral evidence is produced and reproducible on both sides.

1.99.1	src/	0
rustc / cargo verified 2026-10-29	artifact home — not docs/	external crates permitted

3.3 The Declared Shape of the First Artifact Increment

Element	Declared content
File	`src/qnir_assemble.rs` — Rust, standard library only, single file compiled with plain `rustc` (no manifest), preserving one-file-at-a-time
Version bindings	Cites §4.2 (F7 share); binds the semantics version to `qnir.md` v0.3.0-draft.4 bytes; declares the encoding version pinning §2.6's layout as committed; DRAFT status
Source-form grammar	Deterministic, byte-exact syntax for the nine frozen families, value literals with declared bounds, block structure, EXT side table, profile reference — the grammar is this wave's declared deliverable and amends nothing in §1–§3
Declared corpus	Source forms of the normative examples over the frozen core, provenance noted per example; scope fixed at the wave, citable by repository bytes; inexpressible examples routed to extension-boundary classification, never forced
Determinism evidence	Self-test mode: double-encode byte-equality assertion; corpus re-encoding emits byte-exact §2.6 encodings
Companion budget	Zero companions, declared with reason: tools are not portal pages; no navigation, glossary, or index edits

Table 4 — Deliverable shape of the first artifact increment (adjudicated; awaiting review outcome)

One design consequence is declared now so the reviewer can audit it against the committed spec: §2.7 requires values to be exact integers of unbounded range, and Rust's standard library has no arbitrary-precision integer.

The artifact therefore implements minimal magnitude arithmetic over sequences of 64-bit words — which is not a workaround but precisely §2.6's own value encoding, the minimal big-endian sequence of 64-bit magnitude words. The bignum is the encoding, the implementation adds no external dependency, and the supply-chain surface of a normative artifact stays empty. The forward order of the remaining artifact share is registered as assembler, validator, committed suite, reference interpreter, second implementation — each wave separately adjudicated — with the suite's expected outcomes to be derived from committed spec bytes, never from any implementation, and the second implementation in a different language than the reference, per the independence declaration of §4.1.

3.4 Sequencing with Loop 3

The Loop-3 gate — a completed v0.3 draft at the Steps 1–3 minimum — has been met since v0.3.0-draft.3 and is recorded in the standing notes of `qnir.md` §5; whether and when the loop opens is the review program's own act. When the principal proposed running the review before any artifact is drafted, the adjudication was honestly re-opened rather than defended, because the principal's act changes the option space: review-first is π-positive on all three axes. Efficiency, because a specification defect caught now costs one review while the same defect caught after four artifacts inherit it costs roughly five-fold rework; Modularity, because the artifacts then build on reviewed, stable bytes; Portability, because both implementations would cite independently reviewed text, which strengthens the cross-implementation independence claim itself. This document is the act of arming that review: Chapter 4 carries the substantive open question, and Chapter 6 states exactly what the review is asked to produce.

4. The Open Question: The Seed-Language BOC before QnLang Design

4.1 What a Seed Is, and Why the Question Precedes QnLang

The principal directs this question at the smallest seeds: the minimal executable realizing the nine frozen families of `qnir.md` §1.2 — the minimum QnLang instructions — from which the full QnLang extends through recursive generational steps, each generation priced through the deferral register of §1.3 and admitted only with evidence, as Step 6 of the repair sequence requires. The seed is therefore not a prototype to be discarded but the first rung of a bootstrap ladder: seed to primitives, primitives to the full language, with every later generation inheriting the substrate conventions the first rung fixed — word width, endianness, the arithmetic model, and the encoding discipline of §2.6. This inheritance structure is why the language question cannot wait for QnLang's design: the surface language's compiler and toolchain targets, and the entire generational ladder built on them, stand on conventions chosen before the first primitive exists.

Three committed facts make the question adjudicable now rather than speculative. First, the Step-4 oracle artifacts are the first binaries the program must actually produce, and they bind §2.6's encodings — the seed language is the language of the assembler, the validator, and the reference interpreter before it is anything else.

Second, the promotion criterion requires independent differential execution, and §4.1 requires the two implementations to be separate codebases with no shared code — so at least two seeds in different languages must ultimately exist, and the reference seed's language fixes the contrast class the second seed must provide. Third, the byte-level protocol is unforgiving of substrate nondeterminism: equal inputs must produce byte-equal outputs, so the language, its toolchain, and its runtime all enter the determinism claim that `Eval(Intent, D, V) = Execution` makes.

4.2 Governing Criteria, Derived from the Committed Specification

Under the No Implicit Rule discipline, the criteria for this adjudication are derived from committed sections rather than from taste. Seven criteria suffice, each citing its binding section: determinism (C1) from §2.6 and §4.2; exact unbounded arithmetic (C2) from §2.7, which fixes values as exact integers of unbounded range while the transport stays a minimal big-endian sequence of 64-bit magnitude words; static legality (C3) from §4.2's four declared rejection classes; differential independence (C4) from §4.1's no-shared-code declaration; artifact portability (C5) from the promotion criterion's requirement that claims be checkable against repository bytes wherever the program runs; the generational bootstrap path (C6) from the sequence's own inheritance norm; and the trusted-computing-base size (C7) from the fact that the conformance oracle is itself part of the trust base its verdicts defend. The table states each criterion as a checkable question.

#	Criterion	Checkable question	Derived from
C1	Determinism	Do equal inputs yield byte-equal outputs across machines and runs, with no hash-seed, locale, time, or undefined-integer leakage?	§2.6, §4.2
C2	Exact arithmetic	Can exact unbounded integers be realized conformantly — ideally as the 64-bit magnitude-word encoding itself — without external dependencies?	§2.6, §2.7
C3	Static legality	Can the four Intent-rejection classes and the static operand bounds be enforced mechanically with typed, auditable checks?	§2.1, §3.2, §4.2
C4	Differential independence	Does the choice structurally forbid shared code with the second implementation, which must be a different codebase?	§4.1
C5	Artifact portability	Do the seed binaries run wherever the program needs evidence — the repository's CI, the principal's machines, future substrates?	promotion criterion
C6	Generational bootstrap	Can the seed host the recursive generations toward full QnLang, fixing substrate conventions once and stably?	sequence order norm
C7	Trusted computing base	How much runtime and toolchain stands below the seed's verdicts, and how auditable is it?	§4 oracle purpose

Table 5 — Adjudication criteria derived from committed sections

4.3 The Candidate Set and Preliminary Input

The principal names three candidates and invites the review to extend the set: Rust; WAT, the text form of WebAssembly, targeting wasm/wasi; and lower assembly codes. What follows is the drafting agent's preliminary input, explicitly marked as input for adjudication and not as the adjudication — the reviewer's verdict governs before the first artifact increment is drafted. Each candidate is stated with its strongest case, its costs against the criteria, and the committed-spec interactions the review should verify.

Rust. Memory safety without a garbage collector, checked arithmetic available at every site, and a toolchain of record already verified in this program's workspace (rustc/cargo 1.99.1, 29 October 2026). Native compilation puts the differential-execution hot path at machine speed, and the exact-arithmetic requirement of C2 is met conformantly: the standard library has no bignum, so the artifact implements minimal magnitude arithmetic over 64-bit words — which is §2.6's own value encoding — adding no external crate and keeping the supply-chain surface of a normative artifact empty. A single `rustc`-compiled source file preserves one-file-at-a-time literally, and static, cross-compilable binaries serve C5; the same toolchain can additionally emit wasm targets, which keeps a bridge to the portability profile of the second candidate. Costs: the rustc version must be declared in the artifact's version bindings, and the language's own complexity sits in C7's trusted base, bounded here by the small, dependency-free shape of the artifact.

WAT / wasm-wasi. The seed's binary format would itself be a WebAssembly module: maximally portable binaries (C5 at its strongest — any conformant runtime executes them), sandboxed execution, and a validation discipline specified with formal precision, which is an instructive precedent for this program's own validator. The core wasm integer semantics are deterministic, and the frozen core contains no floating point, so the classic nondeterminism concerns do not arise; WAT as the text form even parallels the normative assembler's source-form role — a human-writable syntax over a canonical binary. Costs: wasm offers only 32- and 64-bit integers, so C2's exact unbounded arithmetic must be layered over i64 magnitude words with synthesized carries — implementable, and still aligned with §2.6's 64-bit magnitude words, but more code on the hot path; the runtime is an additional trusted-computing-base layer below the seed (C7); wasi's preview versioning churns under the seed; and the host runtime contract becomes part of the declared version bindings whether or not the program controls it.

Lower assembly. The minimal trusted base and the maximal per-instruction auditability: every byte of the seed is what it appears to be, and C7 is at its strongest. But the costs run against the program's actual needs: per-architecture ports multiply the seeds and the differential surface the protocol must cover; no memory safety assists C3; bignum arithmetic, static-legality checking, and encoding discipline are all hand-rolled at the highest maintenance tier; and artifact portability (C5) is the weakest of the three, since each seed binary is bound to one instruction set. Assembly remains the right home for future node-substrate work — which is precisely where the standing governance queue already banks it, behind Q4's trigger, rather than in the software oracle's path.

Criterion	Rust	WAT / wasm-wasi	Lower assembly
C1 Determinism	High — defined integer semantics, checked arithmetic	High — core spec deterministic; no floats in core	High in principle; nothing assists it
C2 Exact arithmetic	High — magnitude words are §2.6's own encoding	Medium — layered over i64 with synthesized carries	Low — fully hand-rolled
C3 Static legality	High — typed, checked constructions	High — validator precedent is wasm's own	Low — no assist
C4 Independence	High — second impl in another language	High — runtime boundary aids separation	High — but multiplies ports
C5 Portability	High — static, cross-compilable; wasm target possible	Highest — any conformant runtime; wasi churn	Low — per-architecture
C6 Bootstrap	High — stable toolchain, self-hosting path	High — hardware-independent substrate	Medium — stable but per-ISA
C7 Trusted base	Medium — rustc and std below the artifact	Medium-low — runtime layer below the module	Highest — nothing below but the machine

Table 6 — Preliminary drafter's scoring against C1–C7 (input for adjudication, not the adjudication)

4.4 The Questions Put to the Review

The drafting agent proposes the following questions for the open-question registry, suggesting the numbering continuity QO-33 onward (QO-16 through QO-32 being consumed by Loop 2); the reviewer may adopt or renumber at its discretion. First, the seed-language BOC itself: adjudicate the implementation language for the reference seed and the oracle artifacts — Rust, WAT/wasm-wasi, lower assembly, or any candidate the review adds — under criteria C1 through C7, before the QnLang design begins. Second, the generational ladder: state which invariants each recursive generation must preserve — the nine-family core, the §2.6 encodings, the ledger equation, the version-tuple discipline — so that extension by generations cannot silently drift from the freeze. Third, the second implementation's contrast class: which different language maximizes the independence value of the differential protocol, and whether wasm/wasi serves better as the second seed's substrate or as a future promotion substrate under Q4's trigger.

The drafting agent's preliminary ordering — Rust for the reference artifacts, a second implementation in a different language, assembly reserved for node-substrate work behind Q4 — is submitted as input, and the program binds itself in advance to the reviewer's verdict: if the review returns a different BOC, the declared shape of Table 4 is revised to the adjudicated language before `src/qnir_assemble.rs` or any successor filename is drafted, and the wave record will cite the review's bytes for the change. This is the same discipline that bound the v0.3 draft to Loop 2's repair sequence, applied one level down: the principal and the drafting

agent choose the candidates and fix the criteria, and the independent review adjudicates.

5. Governance State and Scope of This Submission

5.1 The Standing Governance Queue

Six trigger-gated governance entries are banked in `docs/index.md` v1.14.0 under the BOC policy, and the discipline of the queue is that nothing in it is rejected and nothing is scheduled: each entry admits to design only when its trigger fires, and the wave that fires a trigger prices the design before the design begins. None has fired. The entry most easily mistaken as fired by this submission's subject matter is Q4, the cdqn Node Contract and Anchor Re-instantiation Sequence: its trigger reads "QnOS/node design start, or QnIR v0.3 promotion requiring a reference execution substrate," and the committed standing notes of `qnir.md` §5 record the rationale — the Step-4 conformance oracle, its specification share and its artifact share alike, is a software oracle, not a node substrate. The seed-language question of Chapter 4 is likewise a software-oracle decision and does not pre-adjudicate Q4: if the review selects wasm/wasi for its portability profile, that remains a choice about seeds and binaries, not the five-capability substrate contract Q4 prices.

#	Entry	Trigger	Status
Q1	Storage Semantics and Residency Contract	First legacy-conversion wave or first mass-witness production	QUEUED
Q2	Anchor Governance Package	First pending crystallization, first fracture exercise, or any anchor contract requiring a v2 designation	QUEUED
Q3	Anchor Evidence Residency Classing	Rides the Q1 wave (same trigger)	QUEUED
Q4	cdqn Node Contract and Anchor Re-instantiation Sequence	QnOS/node design start, or QnIR v0.3 promotion requiring a reference execution substrate	QUEUED
Q5	Grounding-Class Sealing	Deterministic measurement runs available	QUEUED
Q6	Glossary Wave III	Any canon file referencing a listed candidate term	QUEUED

Table 7 — Standing Governance Queue (verbatim-condensed from index.md v1.14.0; all entries QUEUED)

5.2 Review Program State and the Draft's Status

Two loops of the Adversarial Review Program are complete, as recorded in Chapter 1, and the Loop-3 gate — a completed v0.3 draft at the Steps 1–3 minimum — has been met since v0.3.0-draft.3. This document, together with the repository dump the principal supplies, constitutes the Loop-3 submission; commissioning the review remains the principal's act, and the drafting agent opens no loop by any increment. The draft's own status is unchanged by this submission: `qnir.md` remains v0.3.0-draft.4 with DRAFT status, QnIR remains [PENDING] in the portal milestone, and the promotion criterion quoted in Chapter 2 alone governs canon entry. The milestone and the promotion criterion are restated nowhere but by citation, and no canon file is edited by this document.

5.3 What This Submission Changes, and What It Does Not

This PDF is a new additive artifact proposed for `docs/AiReviews/`, following the naming convention of its retired predecessor, and it mutates nothing: not `qnir.md`, not the portal index, not the navigation registry, not the glossary. Its companion budget is zero and declared with the reason that a review submission is not a portal page; if the portal later registers it, that is a future one-line additive wave, declared here so that no unbudgeted-companion flag can recur. What the submission does change is the program's option space: it converts the Loop-3 opening from an available act into an armed one, it puts the seed-language question to independent adjudication before QnLang design, and it fixes in advance the program's binding to the verdict — the artifact share proceeds on reviewed bytes, in the reviewed language, or it proceeds behind the review's repair sequence if findings require one.

6. What Is Asked of the Loop-3 Review

6.1 Scope of Review

The review's scope is the repository dump the principal supplies, this derivation document as its narrative map, and four specific objects within them. First, the committed repair-sequence increments: `qnir.md` §1 through §4 at v0.3.0-draft.4, audited against the sequence's acceptance conditions and against the promotion criterion. Second, the four forced single-token edits that v0.3.0-draft.4 declares in its wave record — the two forward-map cross-references, the standing-queue cross-reference, and the semantics-version self-reference — verified to be bookkeeping only, with no rule, encoding, or register changed. Third, the repository state around the draft: the portal index, the navigation registry, the glossary entries the draft uses under include parity, and the cross-file dependency declarations. Fourth, the open questions of Chapter 4, with the seed-language BOC first among them.

6.2 Output Convention

The program's established convention is that the reviewer works in its own numbered series: findings as F-numbered entries, answers in the open-question registry, a verdict in the pattern of Loop 2, and — if findings require one — a repair sequence whose order is itself normative. Continuity is proposed, not imposed: the review may number its findings F9 onward and its answers QO-33 onward, or open its own series, as it judges. The returned artifact of record is committed verbatim as `docs/AiReviews/AiReviewQnIRv0.3.md`, as Loop 2's was, so that the review's bytes join the repository the program commits against. If the verdict returns findings, the repair sequence governs and the artifact share sequences behind its discharge; if the verdict is a clean bill, the first artifact increment proceeds on reviewed bytes in the adjudicated language.

6.3 Independence and the Limitations Statement

The reviewer is an independent model, not the drafting agent, and the value of this loop is exactly that independence: a drafter reviewing its own derivation would reproduce its own blind spots, which is the failure mode the v0.2 repair sequence exists to correct. For the same reason, the program asks the review to state precisely what it did and did not independently reproduce — Loop 2's limitation statement is the template — so that the promotion criterion's checkable claims inherit no unverified assumptions from this submission. Where the review reproduces a claim against repository bytes, saying so makes the claim stronger; where it cannot, saying so marks the claim for the artifact waves' behavioral evidence instead of leaving the boundary silent.

6.4 After the Verdict

Either outcome leaves the program's main line, its governance queue, and its promotion criterion unchanged in form. With findings, the repair sequence joins the forward map and the artifact share re-sequences behind it, exactly as Loop 2's sequence re-ordered the v0.3 draft. With a clean bill, the adjudicated shape of Table 4 stands and the first artifact increment — `src/qnir_assemble.rs` or its adjudicated successor — is drafted on reviewed bytes, its behavioral evidence produced and re-executed on both sides under the execution-verification protocol of Chapter 3. In both cases the next report back to the repository is a committed wave record citing the review, so that the chain from verdict to artifact to evidence stays unbroken and citable — which is the state the promotion criterion has demanded since Loop 2, and the state this submission exists to reach.

References

Nguyen, C. D. Q. (2026). QnIR — The Intermediate Representation, v0.3 working draft, v0.3.0-draft.4. Repository: docs/qnir.md — §1 the frozen minimal core; §2 operational semantics; §3 work accounting; §4 the conformance oracle; §5 the forward map and standing-queue notes.

Nguyen, C. D. Q. (2026). From the Qn Universe to QnIR v0.2 — Loop-2 adversarial review (Kimi K3.1-Pro, 27 October 2026). Repository: docs/AiReviews/AiReviewQnIRv0.2.md — findings F1–F8, answers QO-16–QO-32, the 16-objection grading table, the v0.3 repair sequence (§5), and the promotion criterion.

Nguyen, C. D. Q. (2026). Portal index, v1.14.0. Repository: docs/index.md — the Standing Governance Queue (Q1–Q6), the Adversarial Review Program record (Loops 1–2), and the QnIR [PENDING] milestone.

Nguyen, C. D. Q. (2026). Site navigation registry. Repository: docs/_data/navigation.yml — the QnIR Draft entry and its budgeted description extensions.

Nguyen, C. D. Q. (2026). Simplicity and Empiricism — the constitutional specification, v1.3.0. Repository: docs/simemp.md — §7 the BOC policy; the Layer-0 hardware telemetry envelope.

Nguyen, C. D. Q. (2026). Abstraction Layers, v1.9.0. Repository: docs/abstractionLayers.md — §6.3 and §13 the QnIR positioning and the fixed-width deterministic machine description reconciled by qnir.md §2.7.

Nguyen, C. D. Q. (2026). Primitive terms glossary. Repository: docs/_data/glossary/primitives.yml — the qnir, boc-policy, no-implicit-rule, and related term definitions used under include parity.

Nguyen, C. D. Q. (2026). Qexpr — the QnLang surface expression layer, v1.4.0. Repository: docs/qexpr.md — the collapse relationship the QnIR layer inherits.

Nguyen, C. D. Q. (2026). From the Qn Universe to QnIR v0.2, draft v0.2.2 — the retired 187-page derivation PDF (docs/AiReviews/Qn-Universe_to_QnIR-v0.2_Derivation_DRAFT-v0.2.2.pdf), reviewed by Loop 2 and retired after verbatim transcription; retained as the historical record of the submission shape this document deliberately replaces.